

RAG against the machine

Explicação passo a passo do código e do processo de execução

Documento de estudo criado para explicar, ficheiro a ficheiro e linha a linha, como funciona o sistema de Retrieval-Augmented Generation (RAG) que responde a perguntas sobre o repositório do vLLM, e qual é a ordem exata de execução do projeto.

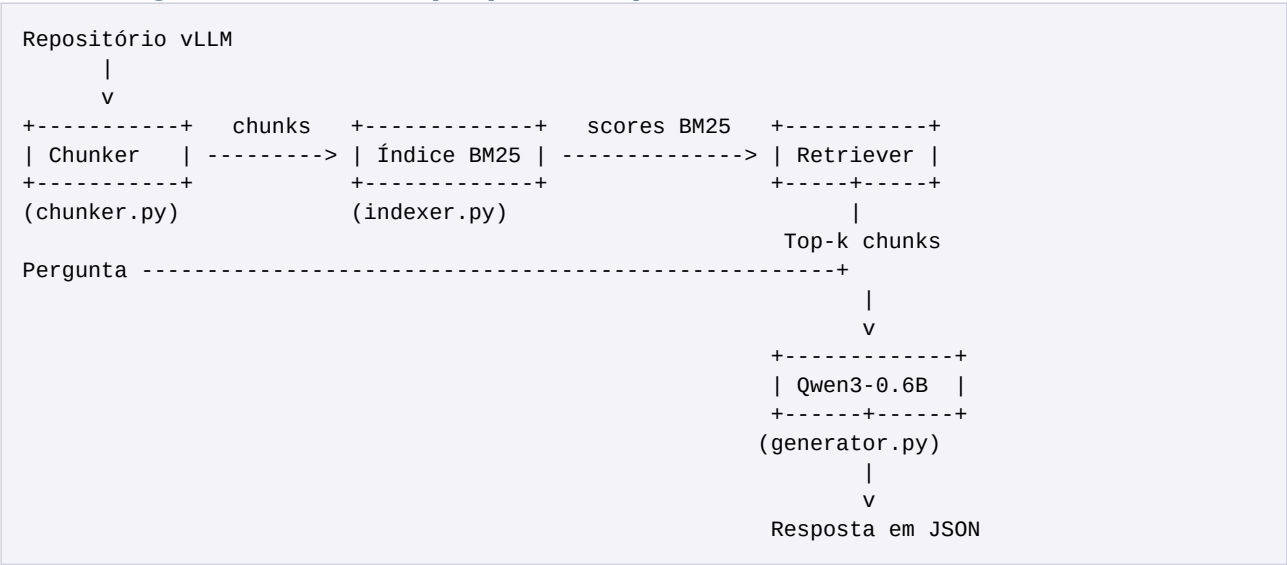
1. Visão geral do projeto

Este projeto constrói um sistema **RAG (Retrieval-Augmented Generation)**: um sistema que, em vez de responder a perguntas apenas com o "conhecimento interno" de um modelo de linguagem, primeiro **procura** as partes mais relevantes de uma base de código/documentação (o repositório vLLM) e só depois pede a um modelo de linguagem (LLM) que gere uma resposta **baseada nesse contexto encontrado**. Isto reduz alucinações e permite responder com precisão sobre um código-fonte específico que o modelo nunca viu durante o seu treino.

O sistema tem duas grandes fases:

- **Fase de indexação (offline):** o repositório vLLM é dividido em pedaços de texto ("chunks") e esses pedaços são organizados num índice de pesquisa (BM25), para poderem ser encontrados rapidamente.
- **Fase de pergunta/resposta (online):** dada uma pergunta do utilizador, o sistema pesquisa no índice os chunks mais relevantes e envia-os, junto com a pergunta, a um modelo de linguagem (Qwen3-0.6B) que gera a resposta final.

1.1. Diagrama do fluxo (arquitetura)



1.2. Papel de cada ficheiro

Ficheiro	Responsabilidade
src/student/chunker.py	Divide ficheiros de código (.py) e de texto/Markdown em pedaços ("chunks") de tamanho controlado.
src/student/indexer.py	Percorre o repositório, aplica o chunker, tokeniza o texto e constrói/guarda/carrega o índice de pesquisa BM25.
src/student/generator.py	Carrega o modelo Qwen3-0.6B e gera a resposta em linguagem natural a partir da pergunta e dos chunks relevantes.
src/student/models.py	Define os modelos de dados (Pydantic) usados para validar e estruturar os ficheiros JSON de entrada e saída.
src/student/__main__.py	Ponto de entrada da linha de comandos (CLI): liga tudo — index, search, answer, evaluate, e test.
Makefile	Automatiza os comandos (instalar, preparar dados, indexar, testar, gerar respostas).

2. Fluxo de execução, passo a passo

Esta secção explica a ordem exata em que os comandos são corridos e o que acontece "por trás das cortinas" em cada um. É a sequência que deves saber explicar do início ao fim.

Passo 1 — make install

Corre `uv venv` e `uv sync`. Cria um ambiente virtual Python e instala todas as dependências definidas em `pyproject.toml` (`fire`, `pydantic`, `rank-bm25`, `tqdm`, `transformers`, `torch`, `accelerate`, `flake8`, `mypy`). No fim, o pacote 'student' fica instalado e disponível para ser corrido com `uv run python -m student`.

Passo 2 — make prepare

Extrai o `vllm-0.10.1.zip` para `data/raw/` (este é o código-fonte real do vLLM que vai ser indexado) e extrai também os datasets públicos de perguntas/respostas para `data/datasets/AnsweredQuestions/` e `data/datasets/UnansweredQuestions/`. Também trata, opcionalmente, de preparar materiais privados de exame (`moulinette`) se estes existirem localmente.

Passo 3 — make index (chama 'student index')

Aqui entra o coração do sistema. O comando cria um objeto `BM25Index` (`indexer.py`) apontando para `data/raw/vllm-0.10.1`, chama `idx.build()`, que: (1) percorre recursivamente todos os ficheiros do repositório, ignorando pastas técnicas (`.git`, `__pycache__`, `node_modules`, etc.); (2) para cada ficheiro válido (`.py`, `.md`, `.rst`, `.txt`, `.yaml`, `.yml`, `.toml`, `.cfg`, `.ini`, `.sh`), chama o `chunker.py` para o dividir em pedaços ("chunks"); (3) tokeniza cada chunk (função `tokenize()`); (4) constrói um índice `BM25Okapi` com todos os chunks tokenizados. No final, o índice é guardado em disco (`data/processed/bm25_index/`) para não ser preciso reconstruí-lo em cada execução.

Passo 4 — pesquisa (comando 'search' ou 'search_dataset')

Dada uma pergunta em texto, o índice `BM25` é carregado do disco, a pergunta é tokenizada da mesma forma que os chunks foram, e o `BM25` calcula uma pontuação de relevância entre a pergunta e todos os chunks indexados. Os `k` chunks com maior pontuação são devolvidos, cada um identificado pelo seu ficheiro de origem e pelo intervalo de caracteres (início/fim) dentro desse ficheiro.

Passo 5 — geração de resposta (comando 'answer' ou 'answer_dataset')

Com os chunks mais relevantes já encontrados, o `generator.py` carrega o modelo `Qwen/Qwen3-0.6B` (a partir do HuggingFace), monta um "prompt" que inclui o texto real desses chunks (lido diretamente do repositório usando os índices de caracteres) e a pergunta, aplica o template de conversa do modelo (ChatML) e pede ao modelo para gerar uma resposta curta, factual e baseada apenas nesse contexto.

Passo 6 — evaluate

Compara os resultados de pesquisa do sistema com um "gabarito" (ground truth) que já contém as fontes corretas para cada pergunta. Calcula a métrica **Recall@k**: a proporção das fontes corretas que o sistema conseguiu encontrar entre as top-k que devolveu, usando uma medida de sobreposição de intervalos de caracteres chamada IoU (Intersection over Union).

Resumo dos comandos, na ordem em que se executam:

```
make install          # cria venv e instala dependências
make prepare          # extrai o vLLM e os datasets
make index            # constrói o índice BM25 (chunking + BM25Okapi)
make test             # search_dataset + evaluate (Recall@k)
make answer_dataset   # gera respostas do LLM para as perguntas
make lint             # flake8 + mypy
make deep-clean       # limpa tudo antes de submeter
```

3. models.py — os modelos de dados (Pydantic)

Este ficheiro não tem lógica de negócio; define apenas a **forma** (schema) dos dados que circulam no sistema, usando a biblioteca **Pydantic**. Isto garante que os ficheiros JSON de entrada e de saída têm sempre a estrutura esperada, validando automaticamente tipos e campos obrigatórios.

class MinimalSource

Representa uma "fonte": um ficheiro (file_path) e o intervalo de caracteres nesse ficheiro (first_character_index e last_character_index) onde está a informação relevante. É a unidade básica de "onde encontrámos a resposta".

class UnansweredQuestion

Uma pergunta simples: tem um question_id (gerado automaticamente com uuid4 se não for fornecido) e o texto da question. É o formato usado nos datasets sem resposta, onde o sistema tem de descobrir tudo sozinho.

class AnsweredQuestion

Herda de UnansweredQuestion e acrescenta a lista de sources (as fontes corretas, o "gabarito") e o texto da answer esperada. É usada nos datasets de treino/avaliação.

class RagDataset

Um conjunto de perguntas (rag_questions), cada uma podendo ser uma AnsweredQuestion ou uma UnansweredQuestion (tipo União). É o formato dos ficheiros de dataset carregados do disco.

class MinimalSearchResults

O resultado da pesquisa para uma pergunta: o seu id, o texto da pergunta e a lista de retrieved_sources (as fontes que o sistema encontrou).

class MinimalAnswer

Herda de MinimalSearchResults e acrescenta o campo answer — a resposta gerada pelo LLM.

class StudentSearchResults

Agrupa todos os MinimalSearchResults de um dataset inteiro, junto com o valor de k usado na pesquisa. É o formato de saída do comando search_dataset.

class StudentSearchResultsAndAnswer

Igual ao anterior, mas a lista de resultados é composta por MinimalAnswer (já com respostas geradas). É o formato de saída do comando answer_dataset.

4. chunker.py — a divisão do texto em pedaços ("chunks")

Antes de qualquer pesquisa ser possível, o texto tem de ser dividido em pedaços mais pequenos ("chunks") — não faz sentido indexar ficheiros inteiros como uma única unidade, porque uma pergunta normalmente só é respondida por uma pequena parte de um ficheiro. Este módulo usa **duas estratégias diferentes**, dependendo do tipo de ficheiro.

4.1. get_file_chunks() — o ponto de entrada

É a função chamada pelo indexador para cada ficheiro. Olha para a extensão do ficheiro: se for `.py`, usa `chunk_python_file()` (chunking inteligente baseado na estrutura do código); para qualquer outra extensão de texto, usa `chunk_text_file()` (chunking por parágrafos).

4.2. chunk_python_file() — chunking baseado em AST

Esta é a parte mais sofisticada do chunker. Em vez de cortar o código a cada N caracteres (o que podia partir uma função a meio, tornando o pedaço incompreensível), usa o módulo `ast` do Python (Abstract Syntax Tree) para **compreender a estrutura do código**:

- `ast.parse(content)` transforma o código-fonte numa árvore que representa a sua estrutura (funções, classes, etc.).
- O código percorre essa árvore (`ast.walk`) à procura de nós do tipo `FunctionDef`, `AsyncFunctionDef` ou `ClassDef` — ou seja, definições de funções e classes de topo.
- Para cada uma dessas definições, calcula o índice de caractere onde ela começa (a partir do número de linha, `node.lineno`), e guarda esse índice como uma "fronteira" (boundary) de chunk.
- As fronteiras são ordenadas e cada intervalo entre duas fronteiras consecutivas torna-se um chunk — assim, cada função ou classe fica inteira dentro do seu próprio chunk (a não ser que seja maior do que `max_chunk_size`, caso em que é subdividida).
- Se o ficheiro tiver um erro de sintaxe (`SyntaxError`) e o `ast.parse` falhar, o código recorre ao método de recurso (fallback): `chunk_text_file()`, tratando o ficheiro como texto simples.

4.3. chunk_text_file() — chunking por parágrafos

Usado para ficheiros de texto/Markdown (e como recurso para `.py` inválido). Divide o conteúdo em parágrafos, usando linhas em branco duplas (`\n\n`) como separador. De seguida, junta parágrafos adjacentes de forma gananciosa ("greedy") até ao limite de `max_chunk_size` caracteres, para aproveitar melhor o espaço de cada chunk sem misturar demasiadas ideias diferentes. Parágrafos individuais que já excedem o limite são divididos linha a linha por `_split_by_size()`.

4.4. Funções auxiliares

- `_split_by_size()`: divide um bloco de texto linha a linha, respeitando o limite de tamanho, evitando cortar uma linha a meio sempre que possível.
- `_hard_split()`: o recurso final — corta o texto brutalmente a cada `max_chunk_size` caracteres, usado quando nem sequer uma única linha cabe dentro do limite.
- `_enforce_max_size()`: uma passagem final de segurança que garante que nenhum chunk excede o tamanho máximo, subdividindo à força qualquer chunk que ainda seja demasiado grande.

O tamanho máximo por defeito é **2000 caracteres**, configurável através de `--max_chunk_size` no CLI.

5. indexer.py — construção e pesquisa do índice BM25

Este ficheiro é responsável por transformar os chunks de texto num índice pesquisável, usando o algoritmo **BM25** (uma evolução do TF-IDF muito usada em motores de busca, que tem em conta a frequência dos termos com saturação e a normalização pelo comprimento do documento).

5.1. tokenize() — a tokenização customizada

Antes de qualquer coisa poder ser indexada ou pesquisada pelo BM25, o texto tem de ser convertido numa lista de "tokens" (palavras). Esta função faz mais do que uma tokenização simples:

- Primeiro, o texto é passado para minúsculas e dividido em qualquer sequência de caracteres que não sejam letras/dígitos/underscore (`re.split(r'^\w_+', text.lower())`).
- Para cada token resultante, o código também tenta separar identificadores **camelCase** (inserindo espaço antes de cada letra maiúscula) e identificadores **snake_case** (dividindo por underscore).
- Tanto as sub-partes como o token completo original são adicionados à lista final de tokens.
- Isto significa que uma pesquisa por "openai compatible server" consegue encontrar tanto a classe `OpenAICompatibleServer` como a variável `openai_compatible_server` — o que seria impossível com um tokenizador genérico.

5.2. collect_files() e should_skip_file()

`collect_files()` percorre recursivamente o repositório com `os.walk`, ignorando pastas técnicas listadas em `SKIP_DIRS` (como `.git`, `__pycache__`, `node_modules`, `build`, `csrc`, `benchmarks`) e pastas ocultas. Para cada ficheiro restante, `should_skip_file()` verifica se a sua extensão está na lista `INDEXABLE_EXTENSIONS` (`.py`, `.md`, `.rst`, `.txt`, `.yaml`, `.yml`, `.toml`, `.cfg`, `.ini`, `.sh`) — só ficheiros com essas extensões são indexados.

5.3. classe BM25Index

build() — construção do índice:

- Obtém a lista de ficheiros relevantes com `collect_files()`.
- Lê o conteúdo de cada ficheiro (ignorando erros de leitura/encoding).
- Chama `get_file_chunks()` (do `chunker.py`) para dividir o conteúdo.
- Para cada chunk, guarda um tuplo (caminho, início, fim, texto) em `self.chunks` e adiciona a versão tokenizada do texto ao corpus.
- No fim, constrói o modelo `BM25Okapi(corpus)` da biblioteca `rank_bm25`, que pré-calcula as estatísticas necessárias (frequências de termos, tamanhos médios de documento, etc.) para poder pontuar pesquisas rapidamente mais tarde.

search(query, k) — pesquisa:

- Tokeniza a query da mesma forma que os chunks foram tokenizados (consistência é essencial).
- Chama `self.bm25.get_scores(query_tokens)`, que devolve uma pontuação de relevância BM25 para cada chunk do corpus.
- Ordena os índices dos chunks por pontuação decrescente e seleciona os top-k.
- Ignora chunks com pontuação zero ou negativa (irrelevantes) e remove duplicados exatos (mesmo ficheiro + mesmo intervalo).

- Devolve uma lista de dicionários com `file_path`, `first_character_index`, `last_character_index` e `score`.

save() / load() — persistência do índice:

Como o objeto `BM25Okapi` não é diretamente serializável em JSON, o índice é guardado em três partes: o modelo BM25 em si (`bm25.pkl`, via `pickle`), os metadados de cada chunk — caminho e intervalo — (`chunks_meta.json`) e os textos de cada chunk (`chunk_texts.pkl`, também via `pickle`, por poderem conter caracteres complexos e serem potencialmente grandes). Isto permite recarregar o índice em menos de um segundo, sem ter de reconstruir tudo do zero a cada execução.

6. generator.py — geração de respostas com o LLM

Depois de o índice devolver os chunks mais relevantes para uma pergunta, este ficheiro é responsável por transformar esses chunks numa resposta em linguagem natural, usando o modelo **Qwen/Qwen3-0.6B** (um modelo de linguagem pequeno, com 0.6 mil milhões de parâmetros, leve o suficiente para correr em CPU).

6.1. load() — carregamento do modelo

- Deteta automaticamente o melhor dispositivo disponível: mps (Apple Silicon), cuda (GPU NVIDIA), ou cpu como último recurso — usando `torch.float16` nos dois primeiros casos e `torch.float32` em CPU.
- Carrega o tokenizador e o modelo diretamente do HuggingFace com `AutoTokenizer.from_pretrained` e `AutoModelForCausalLM.from_pretrained`.
- Move o modelo para o dispositivo escolhido, coloca-o em modo de avaliação (`model.eval()`) e desativa o cálculo de gradientes (`torch.set_grad_enabled(False)`), já que não há treino — apenas inferência — o que poupa memória e tempo.

6.2. _resolve_context_path() e _build_prompt()

Os chunks recuperados só guardam o caminho do ficheiro e os índices de caracteres — não o texto em si (para poupar espaço no índice). Por isso, `_build_prompt()` tem de voltar a ler o ficheiro original do disco para extrair o excerto exato (`content[start:end]`), truncado a `max_context_length` caracteres para caber na janela de contexto do modelo. `_resolve_context_path()` tenta várias combinações de caminho (relativo, absoluto, dentro do repo) porque o caminho guardado pode estar em formatos ligeiramente diferentes consoante a máquina onde correu a indexação.

O prompt final combina: uma instrução explícita a dizer ao modelo para usar apenas o contexto fornecido e admitir quando não sabe ("Not enough information in the provided context"), os excertos de código/documentação recuperados, e a pergunta do utilizador.

6.3. generate() — a geração propriamente dita

- Se o modelo ainda não estiver carregado, chama `load()` automaticamente.
- Como o Qwen3-0.6B é um modelo "de conversa" (chat/instruction-tuned), o prompt é envolvido no formato ChatML através de `tokenizer.apply_chat_template()`, com `enable_thinking=False` para evitar que o modelo gaste todo o seu orçamento de tokens a "pensar em voz alta" antes de responder.
- Os tensores de entrada são movidos para o mesmo dispositivo do modelo.
- `model.generate()` é chamado com `do_sample=False` (geração determinística/greedy), `repetition_penalty=1.1` e `no_repeat_ngram_size=4` para evitar respostas repetitivas.
- Só a parte gerada depois do prompt é decodificada de volta para texto (`outputs[0][input_len:]`), removendo os tokens especiais.

7. `__main__.py` — a interface de linha de comandos (CLI)

Este ficheiro liga todos os módulos anteriores e expõe-nos como comandos utilizáveis a partir do terminal, usando a biblioteca **Python Fire** (que transforma automaticamente os métodos de uma classe Python em comandos de CLI, mapeando os parâmetros dos métodos para flags como `-k` ou `--query`).

index

Constrói o índice BM25 a partir de um diretório de repositório e guarda-o em disco. É o comando chamado por `make index`.

search

Recebe uma query de texto, carrega o índice do disco e imprime no terminal os top-k resultados (ficheiro, intervalo de caracteres, pontuação).

search_dataset

Faz o mesmo que `search`, mas para todas as perguntas de um ficheiro de dataset (JSON), guardando os resultados agregados num ficheiro de saída (`StudentSearchResults`).

answer

Pesquisa o índice para uma query, e depois usa o `AnswerGenerator` para produzir e imprimir uma resposta em linguagem natural.

answer_dataset

Lê um ficheiro de resultados de pesquisa já gerado (por `search_dataset`), e para cada pergunta gera a resposta com o LLM, guardando tudo num novo ficheiro JSON.

evaluate

Compara os resultados do sistema com um dataset de gabarito (com as fontes corretas), e calcula o `Recall@k` para $k = 1, 3, 5$ e 10 .

7.1. Funções auxiliares de avaliação

A avaliação de qualidade da pesquisa (`Recall@k`) baseia-se em três funções auxiliares no fim do ficheiro:

- **`_iou(retrieved_start, retrieved_end, gt_start, gt_end)`**: calcula a métrica "Intersection over Union" entre o intervalo de caracteres que o sistema encontrou e o intervalo correto (gabarito) — quanto maior a sobreposição relativa aos dois intervalos juntos, mais próximo de 1.
- **`_source_found(retrieved, gt_source, k)`**: verifica se, entre as top-k fontes devolvidas, existe alguma no mesmo ficheiro que tenha uma sobreposição (IoU) de pelo menos 0.05 com a fonte correta — nesse caso considera-se "encontrada", mesmo que o intervalo exato não coincida perfeitamente.
- **`_compute_recall_at_k(search_results, gt_lookup, k)`**: para cada pergunta do gabarito, calcula a fração das suas fontes corretas que foram "encontradas" pelo sistema, e faz a média sobre todas as perguntas — este valor final é o `Recall@k`.

7.2. Robustez do CLI

Funções auxiliares como `_load_index()`, `_load_json_model()`, `_validate_k()` e `_write_json()` tratam erros de forma amigável — ficheiro não encontrado, JSON inválido, índice em falta, valores de `k` inválidos — imprimindo uma mensagem clara em `stderr` e terminando com código de saída 1, em vez de deixar o programa rebentar com um traceback difícil de interpretar.

8. Decisões de design importantes (para justificar escolhas)

BM25 em vez de TF-IDF

Melhor saturação de termos numa base de código repetitiva, onde palavras como "tokenizer" aparecem centenas de vezes — o BM25 evita que essas repetições dominem artificialmente a pontuação.

Chunking baseado em AST para Python

Dividir nas fronteiras de def/class mantém funções e classes inteiras como unidades de recuperação, em vez de as cortar arbitrariamente a meio.

Junção de parágrafos em Markdown

A documentação do vLLM está naturalmente organizada por linhas em branco; juntar parágrafos relacionados mantém o contexto coeso nos chunks.

Tokenizador customizado (camelCase/snake_case)

Indexar tanto o identificador completo como as suas sub-partes melhora o recall em pesquisas sobre nomes de métodos/classes.

Inferência apenas em CPU (torch.float32)

O Qwen3-0.6B é pequeno o suficiente para correr sem GPU, tornando o sistema mais portátil.

Pydantic v2 para todo o I/O

Garante que os dados que entram e saem do pipeline seguem exatamente os modelos exigidos pelo enunciado, e permite gerar JSON diretamente com `model_dump_json()`.

9. Perguntas típicas que podes ter de responder (e a resposta rápida)

P: Porque é que se usa BM25 em vez de embeddings/vetores semânticos?

R: BM25 é leve, rápido, não precisa de treino nem de GPU, e funciona muito bem em texto técnico onde os termos exatos (nomes de funções, classes) importam mais do que a semântica geral — o que se ajusta bem ao caso de uso de pesquisa numa base de código.

P: Como é que o sistema evita cortar uma função a meio?

R: Usando o módulo `ast` para identificar onde cada função/classe começa no código, e usando esses pontos como fronteiras de chunk.

P: Como é que a resposta do LLM é "ancorada" nos factos (grounded)?

R: O prompt inclui explicitamente o texto dos chunks recuperados e instrui o modelo a responder apenas com base nesse contexto, dizendo que não sabe se a informação não estiver lá.

P: Porque é que o índice é guardado em três ficheiros diferentes?

R: Porque o objeto `BM25Okapi` não é serializável diretamente em JSON; por isso separa-se o modelo (pickle), os metadados (JSON legível) e os textos completos dos chunks (pickle), permitindo recarregar

tudo rapidamente sem reconstruir o índice.

P: O que é o Recall@k e como é calculado?

R: É a proporção das fontes corretas (do gabarito) que aparecem entre as top-k fontes devolvidas pelo sistema, considerando uma fonte "encontrada" se tiver pelo menos 5% de sobreposição (IoU) com a fonte correta no mesmo ficheiro.